

A MINI PROJECT REPORT
ON
BLACK BOX SECURITY TESTING ON THE HOST MACHINE BY
USING OPENVAS

Submitted to Sri Indu College of Engineering and Technology, in partial
fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

IN

CSE- CYBER SECURITY (CSC)

BY

BASA SAIRAJ (20D41A6212)

[REDACTED] (21D41A6201)

[REDACTED] (20D41A6213)

[REDACTED] (20D41A6214)

Under the esteemed guidance of

[REDACTED]

(Asst. prof)



DEPARTMENT OF CSE- CYBER SECURITY (CSC)

SRI INDU COLLEGE OF ENGINEERING & TECHNOLOGY

(An Autonomous Institution under UGC, accredited by NBA, Affiliated to JNTUH)

Sheriguda, Ibrahimpatnam

(2023-2024)

SRI INDU COLLEGE OF ENGINEERING & TECHNOLOGY

(An Autonomous Institution under UGC, Accredited by NBA, Affiliated to JNTUH)

DEPARTMENT OF CSE- CYBER SECURITY (CSC)



CERTIFICATE

Certified that the Mini project report entitled “**Black Box Testing over host machine with the help of OpenVAS**” is a Bonafide work done by **BASA SAIRAJ (20D41A6212)**,
[REDACTED]
[REDACTED] in partial fulfillment of the requirements for the award of the degree of **[REDACTED]** submitted to **Department of Cyber Security, Sri Indu College of Engineering and Technology**, Hyderabad during the Academic Year **2023- 2024**.

MRS. K. SHWETHA REDDY

(INTERNAL GUIDE)

MRS. G. UMA MAHESWARI

HOD (DEPT OF CS/AIML)

PRINCIPAL

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

With great pleasure we want to take this opportunity to express our heartfelt gratitude to all the people who helped in making this project work a success. We thank the almighty for giving us the courage & perseverance in completing the project.

We are thankful to principal Prof. **[REDACTED]**, for giving us the permission to carry out this project and for providing necessary infrastructure and labs.

We are highly indebted to, **[REDACTED]**, Head of the Department of Computer Science and Engineering Cyber Security (CS)& Artificial Intelligence and Machine Learning (AIML), Project Guide, for providing valuable guidance at every stage of this project.

We are grateful to our internal project guide, **[REDACTED]** for her constant motivation and guidance given by her during the execution of this project work.

We would like to thank the Teaching & Non-Teaching staff of Department of Cyber Security & Engineering for sharing their knowledge with us.

Last but not the least we express our sincere thanks to everyone who helped directly or indirectly for the completion of this project

BASA SAIRAJ

20D41A6212

[REDACTED]

[REDACTED]

[REDACTED]

CONTENT

PAGE NO:

ABSTRACT	01
1. INTRODUCTION	
1.1. INTRODUCTION TO PROJECT.....	02
1.2. BLACK BOX SECURITY TESTING.....	02 - 03
1.3. HOST MACHINE.....	03 - 04
1.4. OPENVAS.....	04
1.5. BLACK BOX TESTING WITH OPENVAS.....	05 - 07
2. OBJECTIVE	08 - 09
3. FUTURE SCOPE	10
4. EXISTING SYSTEM AND PROPOSED SYSTEM	
4.1. EXISTING SYSTEM.....	11
4.2. DISADVANTAGES OF USING OPENVAS IN AN EXISTING SYSTEM.	11 - 12
4.3. PROPOSED SYSTEM.....	12 - 13
4.4. ADVANTAGES OF A PROPOSED DEDICATED SYSTEM.....	13 - 14
5. REQUIREMENTS	15 - 16
6. IMPLEMENTATION	
6.1. INTRODUCTION.....	17
6.2. GETTING STARTED WITH OPENVAS.....	17 - 32
7. METHODOLOGY	33
SCAN THE TARGET.....	34 -36
8. RESULT AND CONCLUSION	37 - 42
9. REFERENCES	43
10. GLOSSARY	44
11. APPENDIX	45 - 48

ABSTRACT

This project focuses on the implementation of black box testing techniques using OpenVAS on a host machine. Black box testing is an essential approach to assess the security posture of software applications and system. OpenVAS, a widely used open-source vulnerability assessment tool, provides a comprehensive platform for identifying vulnerabilities from an external perspective.

Throughout this project, we delve into the process of configuring and utilizing OpenVAS to perform black box testing on a host machine. The methodology involves simulating external attacks to uncover potential vulnerabilities, weaknesses, and entry points that malicious actors could exploit. By conducting this testing, we aim to enhance the host machine's security measure and strengthen its resistance to potential cyber threats.

The outcomes of this project offer insights into the effectiveness of OpenVAS as a black box testing tool and highlight its contribution to proactive security assessment. Through practical implementation and analysis, we gain a better understanding of how organizations can safeguard their systems against a variety of cyber risks. Ultimately, this project serves as a valuable resource for individuals seeking to comprehend the significance of black box testing and its practical application using OpenVAS. The project culminates in the generation of comprehensive reports, furnishing organizations with detailed insights into the vulnerabilities identified, their associated risk scores, and actionable recommendations for remediation. This endeavour contributes significantly to bolstering the security posture of host machines and, consequently, the overall cybersecurity resilience of organizations in the face of evolving threats.

INTRODUCTION

Black box security testing with OpenVAS! As technology continues to advance, so do the methods used by cyber criminals to infiltrate systems. It is crucial for organizations to stay ahead of the game and ensure their systems are secure from potential attacks. This is where black box testing comes in.

Black box testing is a method of testing software without knowing its internal structure or coding. OpenVAS is a widely known open-source vulnerability scanner that can be used for black box testing. OpenVAS is from an organization Greenbone Networks. It includes a variety of built in tests and a web interface.

Black box testing is a method used to identify vulnerabilities in software applications. By simulating an attacker's perspective, we can test the system's security measures and identify potential weaknesses. We will focus specifically on black box testing on the host machine using OpenVAS.

BLACK BOX SECURITY TESTING

Black box testing is a method of software testing that examines the functionality of an application without knowing its internal workings. In other words, it's like trying to solve a puzzle without knowing what's inside the box. This approach allows testers to focus on the user experience and behaviour of the application, which can help identify potential vulnerabilities that might be missed with other testing methods.

One example of black box testing is when a tester tries to log into an application using incorrect credentials. If the application allows the login attempt despite the wrong credentials, it indicates a vulnerability that could be exploited by a cyber attacker. By identifying such vulnerabilities, black box testing helps improve the security of software applications.



Black box security testing is a methodology that limits the information. It is also called as closed box testing or external penetration testing. In this technique, the testers often called security experts often called security experts or ethical hackers.

HOST MACHINE

A host machine, also known as a server or a mainframe, is a computer system that provides services or resources to other computers on a network. It is the backbone of any organization's IT infrastructure and contains sensitive information such as customer data and financial records. However, this makes it a prime target for cyber-attacks.

Hackers can exploit vulnerabilities in the host machine's operating system or software applications to gain unauthorized access to the system. Once they have access, they can steal sensitive information, install malware, or even use the machine as a launching pad for further attacks on the network. The consequences of a successful attack on a host machine can be devastating for an organization, leading to loss of revenue, reputation damage, and legal liabilities.



A computer or device that is the target of security testing, which can include servers, workstations, or other networked devices.

OPENVAS

OpenVAS is a powerful tool for black box testing on the host machine. It is designed to identify vulnerabilities in software applications and provide detailed reports on potential threats. With its intuitive interface, OpenVAS makes it easy to scan your system and detect any weaknesses that may be exploited by cybercriminals.

One of the key features of OpenVAS is its ability to perform comprehensive scans of your network. This means that it can identify vulnerabilities not just on the host machine, but also on other devices connected to the same network. In addition, OpenVAS has a large database of known vulnerabilities, which it uses to match against the software installed on your system. This makes it a highly effective tool for identifying both known and unknown vulnerabilities.



Greenbone OpenVAS

Open Vulnerability Assessment Scanner

BLACK SECURITY BOX TESTING WITH OPENVAS

To begin black box testing with OpenVAS, the first step is to select the target host machine and configure the scan settings. This involves setting parameters such as the scan type, port range, and timing options.

Once the scan is initiated, OpenVAS will begin probing the target machine for vulnerabilities. The tool uses a combination of active and passive scanning techniques to identify potential weaknesses in the system. As the scan progresses, the results are compiled into a report that can be used to prioritize remediation efforts.

Black box security testing using OpenVAS (Open Vulnerability Assessment System) involves a systematic approach to identifying vulnerabilities and assessing the security of a target system without any prior knowledge of its internal structure or code. Here are the steps involved in conducting black box security testing with OpenVAS:

1. Preparation and Setup:

- Install and configure OpenVAS on a dedicated testing environment. Ensure that the OpenVAS database and plugins are up to date to cover the latest vulnerabilities.

2. Scope Definition:

- Clearly define the scope of the testing, including the target system or network, IP addresses, and specific services or applications to be tested. Determine what should be included and excluded from the scan.

3. Target Identification:

- Identify the host or network range to be scanned. Ensure you have permission to perform the scan to avoid potential legal and ethical issues.

4. Scan Configuration:

- Configure the OpenVAS scan by specifying the target hosts, ports to scan, and scan type (e.g., full, fast, or specific vulnerability tests). You can also set scan timing options and credentials for authenticated scans if applicable.

5. Initiate the Scan:

- Launch the OpenVAS scan and allow it to run. OpenVAS will perform automated vulnerability assessments and tests against the target system.

6. Scan Execution:

- OpenVAS will scan the target system, checking for known vulnerabilities, misconfigurations, and security weaknesses. It uses its extensive vulnerability database and plugins to identify potential issues.

7. Vulnerability Detection:

- As the scan progresses, OpenVAS identifies vulnerabilities and assigns severity levels based on the Common Vulnerability Scoring System (CVSS). It categorizes findings by potential impact and exploitability.

8. Report Generation:

- OpenVAS generates comprehensive reports that provide detailed information about identified vulnerabilities. Reports include vulnerability descriptions, severity ratings, and recommendations for remediation. These reports are crucial for communicating findings to stakeholders.

9. Risk Assessment:

- Analyse the vulnerabilities identified by OpenVAS and assess their potential risks to the target system or network. Consider factors such as the potential impact on confidentiality, integrity, and availability.

10. Re-Testing and Validation:

- After implementing the recommended remediations, conduct re-testing using OpenVAS to verify that the vulnerabilities have been effectively addressed. Ensure that the fixes do not introduce new vulnerabilities.

11. Documentation:

- Maintain detailed documentation of the testing process, findings, remediation actions, and validation results. Proper documentation is essential for compliance, auditing, and future reference.

13. Continuous Monitoring:

- Implement continuous security monitoring using OpenVAS or other tools to detect and respond to new vulnerabilities and threats as they emerge. Regularly schedule follow-up scans to ensure ongoing security.

14. Knowledge Sharing:

- Share the results and insights gained from the OpenVAS testing with relevant teams and stakeholders within the organization. Foster a culture of security awareness and encourage proactive security practices.

By following these steps, organizations can leverage OpenVAS to conduct effective black box security testing, identify vulnerabilities, and enhance the overall security posture of their systems and networks. This proactive approach helps mitigate security risks and reduces the potential for security breaches and data compromises.

OBJECTIVE

The objective of this project is to demonstrate the effectiveness of Black Box Security Testing using OpenVAS (Open Vulnerability Assessment System) to assess and improve the security posture of a host machine. This project aims to:

- **Assess Vulnerabilities:**

Identify and assess potential vulnerabilities, weaknesses, and security flaws in the target host machine's infrastructure, software, and configurations.

- **Evaluate Attack Surface:**

Analyse the attack surface of the host machine to understand potential entry points for attackers and the impact of vulnerabilities on its security.

- **Perform Black Box Testing:**

Conduct comprehensive black box security testing without prior knowledge of the host's internal structure, codebase, or configurations, simulating the perspective of an external threat actor.

- **Prioritize Vulnerabilities:**

Prioritize identified vulnerabilities based on their severity, potential impact on the host machine, and exploitability.

- **Generate Detailed Reports:**

Create detailed reports that document the vulnerabilities, their risk level, and potential remediation steps, aiding in the decision-making process for mitigating identified security risks.

- **Recommend Mitigation Measures:**

Provide recommendations and best practices for mitigating the identified vulnerabilities, including patching, configuration changes, and security policy improvements.

- **Compliance Assessment:**

Assess the host machine's compliance with relevant security standards and regulations (e.g., CIS benchmarks, GDPR, HIPAA) to ensure adherence to industry best practices and legal requirements.

- **Continuous Improvement:**

Offer guidance on establishing a continuous monitoring and improvement process to enhance the host machine's security posture over time.

- **Knowledge Transfer:**

Facilitate knowledge transfer by sharing insights gained during the testing process with the host machine's administrators, enabling them to better understand and address security concerns.

- **Awareness and Education:**

Promote security awareness and education among stakeholders, emphasizing the importance of regular security testing and proactive security measures.

By achieving these objectives, this project will demonstrate how OpenVAS can be a valuable tool in enhancing the security of host machines by identifying and addressing vulnerabilities from a black box testing perspective, thereby reducing the risk of security breaches and data compromises

FUTURE SCOPE

- Conducting automated vulnerability scans.
- Identifying system weaknesses and misconfigurations.
- Analysing network infrastructure and communication vulnerabilities
- Assessing the effectiveness of security controls
- Detecting emerging threats and zero-day vulnerabilities.
- Evaluating the overall security posture of the host machine.
- **User-Friendly Interface:** Develop a user-friendly web interface for OpenVAS to simplify setup, configuration, and reporting for non-technical users.
- **Advanced Threat Modelling:** Incorporate threat modelling techniques into the testing process to identify potential attack vectors and prioritize testing efforts based on risk analysis.
- **Custom Plugin Development:** Create custom OpenVAS plugins to assess specific applications or devices that are not covered by default, tailoring the testing process to the unique needs of the organization.
- **Enhanced Reporting and Visualization:** Improve the reporting capabilities of OpenVAS by integrating with data visualization tools, making it easier for stakeholders to understand and act on security findings.
- **Machine Learning and AI:** Implement machine learning and artificial intelligence algorithms to enhance the accuracy of vulnerability detection and reduce false positives, making the testing process more efficient.
- **Custom Plugin Development:** Create custom OpenVAS plugins to assess specific applications or devices that are not covered by default, tailoring the testing process to the unique needs of the organization.
- **IoT Security Assessment:** Extend testing to IoT devices and networks.
- **DevSecOps Integration:** Automate security testing within DevSecOps pipelines.

By exploring these future scope areas, the project can stay at the forefront of security testing practices, adapt to evolving technologies, and contribute to the ongoing improvement of security in host machines and connected systems.

EXISTING SYSTEM & PROPOSED SYSTEM

Below, I'll outline how you can perform black box security testing over a host machine using OpenVAS in both an existing system and a proposed system for a project.

EXISTING SYSTEM:

Install OpenVAS: Ensure that OpenVAS is installed on a host machine. You can follow the installation instructions provided on the OpenVAS website.

Configure OpenVAS: After installation, you need to configure OpenVAS. This involves setting up the scanner, creating user accounts, and configuring scan targets (the host machine you want to test).

Scan Configuration: Define the scanning parameters, such as the target IP address or hostname, scan type (full, fast, or custom), and scan schedule (if needed).

Run Scans: Start the vulnerability scans using OpenVAS. The tool will probe the target host for known vulnerabilities, including common software weaknesses, misconfigurations, and security issues.

Analyse Results: Review the scan results generated by OpenVAS. It will provide a list of vulnerabilities, their severity levels, and suggested remediation steps.

Reporting: Create a comprehensive report summarizing the vulnerabilities discovered, their potential impact, and recommended actions to mitigate them. Share this report with relevant stakeholders for remediation.

DISADVANTAGES OF USING OPENVAS IN AN EXISTING SYSTEM:

Complexity: Integrating OpenVAS into an existing system can be complex, especially if the system was not originally designed to accommodate security testing tools. This complexity can lead to configuration issues and potential conflicts with existing components.

Resource Intensive: OpenVAS can be resource-intensive, requiring significant CPU and memory resources to perform scans effectively. If your existing system is resource-constrained, running OpenVAS alongside it may lead to performance issues.

Limited Control: In an existing system, you may have limited control over the underlying infrastructure and security policies. This can hinder your ability to fine-tune OpenVAS settings for optimal security testing.

Integration Challenges: Integrating OpenVAS into an existing system may require custom scripting or development work to make it work seamlessly with your environment. This can be time-consuming and may introduce potential vulnerabilities if not done correctly.

PROPOSED SYSTEM:

If you are proposing a system that incorporates OpenVAS for black box security testing as part of your project, here are the steps you might consider:

System Design: Start by designing the overall system architecture that includes OpenVAS as a security testing component. Identify the components of the system and how they will interact.

Integration: Integrate OpenVAS into the system. You may need to develop scripts or use APIs provided by OpenVAS to initiate scans, retrieve results, and manage scan configurations programmatically.

User Interface: Create a user-friendly interface for users to input scan parameters, schedule scans, and view scan results. This interface can be a web application or a command-line tool, depending on your project's requirements.

Automation: Implement automation features to schedule regular scans, send alerts when critical vulnerabilities are found, and generate automated reports.

Security Measures: Ensure that the system that hosts OpenVAS is itself secure. Apply security best practices, such as regular updates and hardening, to prevent unauthorized access to the vulnerability scanning tool.

Documentation: Create documentation that explains how to use the system, including how to set up and configure OpenVAS for security testing.

Testing: Thoroughly test the system to ensure that it accurately performs black box security testing, generates meaningful reports, and functions as expected.

Deployment: Deploy the proposed system in your project's environment, ensuring it can scan the host machines as intended.

Training: If necessary, provide training to users and administrators on how to use the system effectively.

Maintenance: Regularly update and maintain both OpenVAS and the proposed system to keep them up-to-date with the latest security vulnerabilities and patches.

ADVANTAGES OF A PROPOSED DEDICATED SYSTEM FOR BLACK BOX SECURITY TESTING:

Isolation: A dedicated system for security testing allows you to isolate and control the testing environment completely. This isolation reduces the risk of conflicts with production systems and minimizes the chance of false positives or negatives.

Resource Allocation: You can allocate dedicated resources (CPU, memory, network bandwidth) to the security testing system, ensuring that it can run scans efficiently without impacting the performance of other systems.

Customization: When building a dedicated system, you have full control over its configuration. You can tailor it to meet the specific requirements of your security testing, including custom scan profiles and policies.

Scalability: A proposed system can be designed with scalability in mind, allowing you to expand its capacity as needed to accommodate more extensive testing or a growing number of target hosts.

Automation: Building a dedicated system provides the opportunity to implement automation features for scheduling scans, sending alerts, and generating reports, streamlining the testing process.

Enhanced Security: By designing the system from the ground up with security in mind, you can ensure that it adheres to best practices and is less susceptible to security vulnerabilities itself.

Easier Maintenance: Maintaining a dedicated security testing system is generally easier because you have more control over updates, patches, and system configurations. This reduces the risk of disruptions to other critical systems.

Scalability: A dedicated system can be designed to scale easily as your organization's security testing needs grow. You can add more scanning resources or even implement distributed scanning to cover a broader scope.

Centralized Management: With a dedicated system, you can centralize the management of security testing tools and data, making it easier to track and report on vulnerabilities across your organization.

Compliance: A dedicated system can help with compliance efforts by providing a controlled environment for security testing, which is often required for regulatory and compliance purposes.

In summary, while using OpenVAS in an existing system is possible, it may come with complexities and limitations. On the other hand, a proposed dedicated system offers advantages such as isolation, resource allocation, customization, scalability, and enhanced security. These advantages make a dedicated system an attractive choice for black box security testing in a project, especially when the testing requirements are significant and need to be managed independently.

REQUIREMENTS

Stable Network Connection:

The host machine should have reliable network connectivity to the target host(s) and internet access to fetch vulnerability databases and updates.

A lot of patience ⌚ :

Working with OpenVAS requires patience due to lengthy scans, false positives, complex configurations, and the need to interpret and address vulnerabilities effectively.

System Requirements:

Any operating system running Docker recommended kali Linux or otherwise. Choose a compatible operating system for hosting OpenVAS. Common choices include Linux distributions like Ubuntu, Debian, or CentOS

Minimum requirements:

- 2 CPU Cores
- 4GB RAM
- 20GB Storage

Recommended Requirements:

- 4+ CPU Cores
- 8GB+ RAM
- 60GB+ Storage

OpenVAS Installation:

Install the OpenVAS software suite, including the Greenbone Security Assistant (GSA) for web-based management and the OpenVAS Scanner for vulnerability scanning

Host machine/Target machine:

- Metasploit able
 - Any Webserver
 - Any Website
- Note: you should have legal permissions to scan

Training and Expertise

Skilled Personnel

Network Permissions:

Ensure you have the necessary permissions to scan the target host(s) since scanning without authorization could be illegal.

Target Host Information:

Gather information about the target host(s) you intend to test, including IP addresses, hostnames, and network configurations.

OpenVAS Configuration:

Configure OpenVAS according to your testing requirements. Define scan policies, target hosts, and scan schedules within the OpenVAS setup.

Documentation:

Keep detailed records of your testing procedures, configurations, and findings for later analysis and reporting.

Requirements for conducting black box security testing over a host machine using OpenVAS involve both hardware and software considerations. Meeting these requirements will help ensure the effective and ethical execution of black box security testing using OpenVAS on host machines while minimizing risks and maximizing the benefits of the testing process. Remember to consider the scalability and performance requirements of your project when implementing OpenVAS for black box security testing. Additionally, stay up-to-date with the latest developments in the field of cybersecurity to ensure that your system remains effective in identifying and mitigating vulnerabilities.

IMPLEMENTATION

INTRODUCTION:

OpenVAS is a full-featured vulnerability scanner. Its capabilities include unauthenticated and authenticated testing, various high-level and low-level internet and industrial protocols, performance tuning for large-scale scans and a powerful internal programming language to implement any type of vulnerability test. The scanner obtains the tests for detecting vulnerabilities from a feed that has a long history and daily updates.

OpenVAS has been developed and driven forward by the company Greenbone since 2006. As part of the commercial vulnerability management product family Greenbone Enterprise Appliance, the scanner forms the Greenbone Community Edition together with other open-source modules.

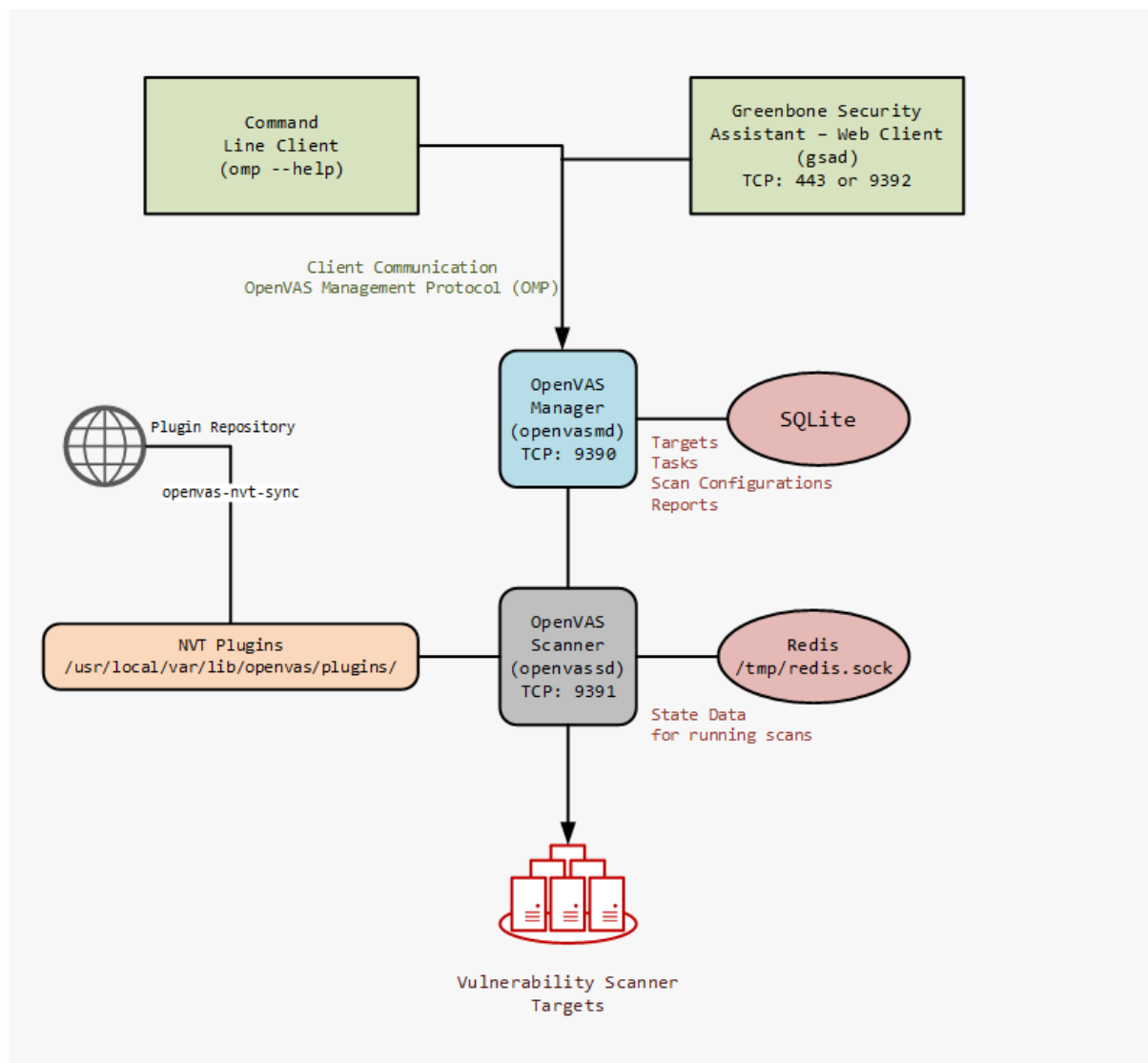
Read more about the history of OpenVAS [Click Here](#)



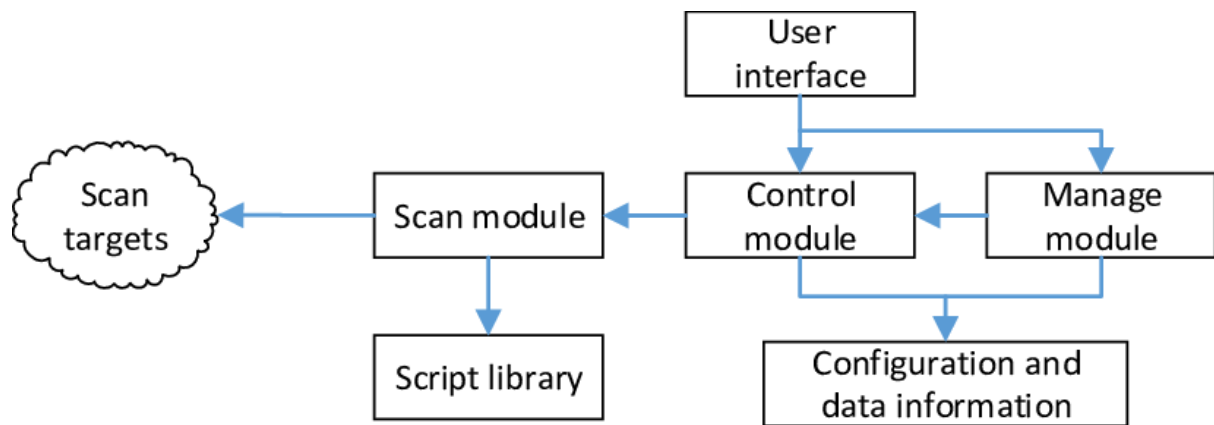
GETTING STARTED WITH OPENVAS:

Overview Of Components

OpenVAS is comprised of a number of services and clients. By understanding the role of the different components you will be much better able to effectively administer OpenVAS servers.



Communication from the client to the OpenVAS Manager is done using the OpenVAS Management Protocol (OMP).



OpenVAS System Architecture Diagram

There are two ways to install OpenVAS:

1. FROM THE SOURCE

Installing from the source can be quite tasking and could even end up still not working in the end. However, using a docker container makes the process simple and almost seamless.

To install OpenVAS and its dependencies on our Kali Linux system run the following command.

```
Sudo apt dist-upgrade -y
```

```
phantom@kali: ~  
File Actions Edit View Help  
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.  
  
(phantom@kali)-[~]  
$ sudo apt dist-upgrade  
Reading package lists ... Done  
Building dependency tree ... Done  
Reading state information ... Done  
Calculating upgrade ... Done  
The following package was automatically installed and is no longer required:  
  gstreamer1.0-pulseaudio  
Use 'sudo apt autoremove' to remove it.  
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

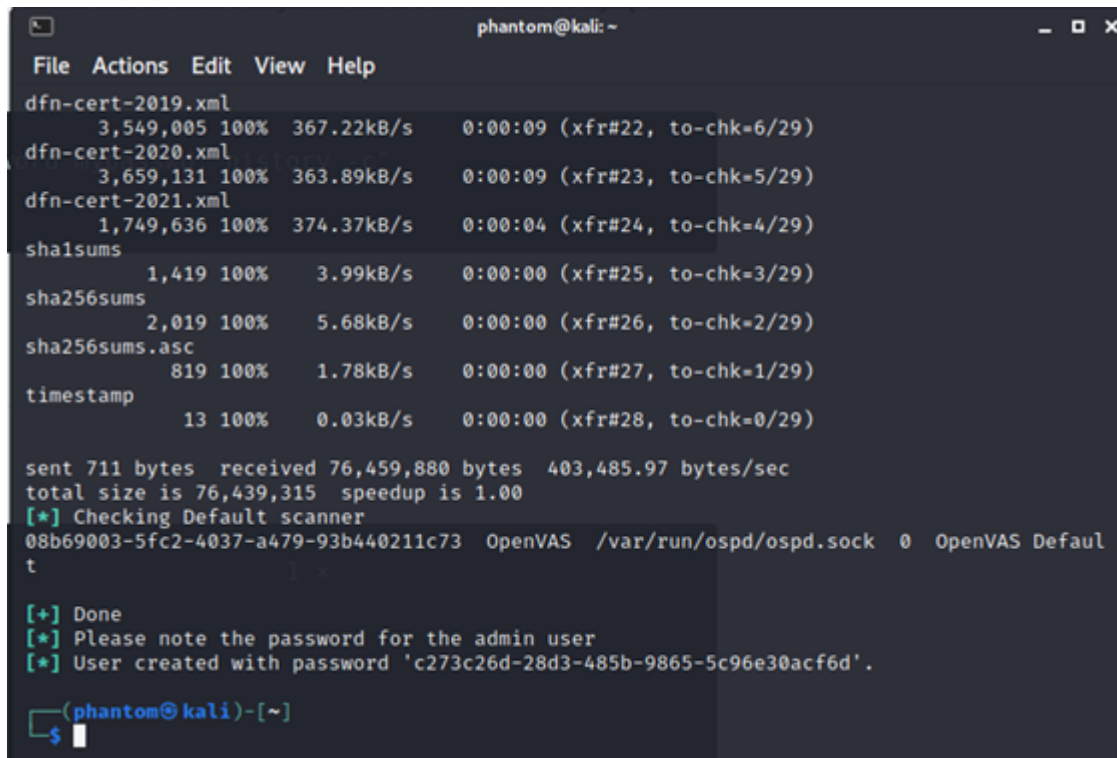
Sudo apt install openvas

```
phantom@kali: ~  
File Actions Edit View Help  
update-rc.d: We have no instructions for the redis-server init script.  
update-rc.d: It looks like a non-network service, we enable it.  
redis-server.service is a disabled or a static unit, not starting it.  
Setting up gvmd (21.4.0-0kali2) ...  
Setting up python3-gvm (21.5.2-0kali1) ...  
Setting up greenbone-security-assistant (21.4.0-0kali1) ...  
greenbone-security-assistant.service is a disabled or a static unit, not starting it.  
Setting up openvas-scanner (21.4.0-0kali2) ...  
Setting up ospd-openvas (21.4.0-0kali1) ...  
ospd-openvas.service is a disabled or a static unit, not starting it.  
Setting up gvm-tools (21.6.0-0kali1) ...  
Setting up gvm (21.4.0.0~kali4) ...  
Setting up openvas (21.4.0.0~kali4) ...  
Processing triggers for fontconfig (2.13.1-4.2) ...  
Processing triggers for kali-menu (2021.2.3) ...  
Processing triggers for desktop-file-utils (0.26-1) ...  
Processing triggers for libc-bin (2.31-12) ...  
Processing triggers for man-db (2.9.4-2) ...  
Processing triggers for mailcap (3.69) ...  
Processing triggers for tex-common (6.16) ...  
Running updmmap-sys. This may take some time... done.  
Running mktexlsr /var/lib/texmf ... done.  
Building format(s) --all.  
This may take some time... done.  
  
(phantom@kali)-[~]  
$
```

The next step is to run the installer, which will configure OpenVAS and download various network vulnerability tests (NVT) or signatures. Due to a large number of NVTs (50.000+), the setting process may take some time and consume a lot of data. In the test setup we used for this tutorial, the complete setup process took 10 minutes, which is not bad.

Run the following command to start the setup process:

```
gvm-setup
```



```
phantom@kali: ~  
File Actions Edit View Help  
dfn-cert-2019.xml      3,549,005 100% 367.22kB/s 0:00:09 (xfr#22, to-chk=6/29)  
dfn-cert-2020.xml      3,659,131 100% 363.89kB/s 0:00:09 (xfr#23, to-chk=5/29)  
dfn-cert-2021.xml      1,749,636 100% 374.37kB/s 0:00:04 (xfr#24, to-chk=4/29)  
shasums                1,419 100% 3.99kB/s 0:00:00 (xfr#25, to-chk=3/29)  
sha256sums             2,019 100% 5.68kB/s 0:00:00 (xfr#26, to-chk=2/29)  
sha256sums.asc         819 100% 1.78kB/s 0:00:00 (xfr#27, to-chk=1/29)  
timestamp              13 100% 0.03kB/s 0:00:00 (xfr#28, to-chk=0/29)  
  
sent 711 bytes received 76,459,880 bytes 403,485.97 bytes/sec  
total size is 76,439,315 speedup is 1.00  
[*] Checking Default scanner  
08b69003-5fc2-4037-a479-93b440211c73 OpenVAS /var/run/ospd/ospd.sock 0 OpenVAS Default  
t  
  
[+] Done  
[*] Please note the password for the admin user  
[*] User created with password 'c273c26d-28d3-485b-9865-5c96e30acf6d'.  
  
(phantom@kali)-[~]  
$
```

After the configuration process is complete, all the necessary OpenVAS processes will start and the web interface will open automatically. The web interface is running locally on port 9392 and can be accessed through **https://localhost:9392**. OpenVAS will also set up an admin account and automatically generate a password for this account which is displayed in the last section of the setup output:

Verify The Installation

You can verify your installation with.

```
gvm-check-setup
```

```

Starting gvmd service
Waiting for gvmd service
OK: gvmd service is active.
Starting greenbone-security-assistant service
Waiting for greenbone-security-assistant service
OK: greenbone-security-assistant service is active.
Step 8: Checking few other requirements...
OK: nmap is present in version 21.04.0-git.
OK: ssh-keygen found, LSC credential generation for GNU/Linux targets is likely to
work.
WARNING: Could not find makensis binary, LSC credential package generation for Mi
crosoft Windows targets will not work.
SUGGEST: Install nsis.
OK: xsltproc found.
WARNING: Your password policy is empty.
SUGGEST: Edit the /etc/gvm/pwppolicy.conf file to set a password policy.

It seems like your GVM-21.4.0 installation is OK.

(phantom@kali)~$

```

Do the same process as the below except instead of default password admin use the above password which is generated when we setup the gvm.

Even you want to do OpenVAS installation from the official (source). You can follow or install from the below link. But we will get dozens of errors regarding database issues etc...

[click Here](#)

2. USING DOCKER CONTAINERS

**NOTE: HERE TO IMPLEMENT THE PROJECT WE USED SECOND APPROACH
DOCKER CONTAINERS**

Installing from the source can be quite tasking and could even end up still not working in the end. However, using a docker container makes the process simple and almost seamless.

A docker container is a lightweight, standalone, executable software package that includes everything needed to run an application, including the code, runtime environment, system tools, and libraries. This allows the application in the container to be deployable almost anywhere.

We don't really need to get into the specifics of things but basically think of it like a Lego set and its box. The Lego set is your application and the box is the container. The box contains the instructions and pieces you need to build the structure.

Likewise, the container has everything you need for deploying your application anywhere. And because it's so versatile, you can create multiple containers to deploy multiple applications without putting much strain on your hardware.

Now that you have an idea of how containers work, let's look at how to install OpenVAS using them.

INSTALLATION STEPS:

We'll start off by opening up a terminal in kali Linux and installing curl and docker.io.

```
sudo apt install curl docker.io
```

Next, install docker.compose

```
sudo apt install python3 python3-pip  
python3 -m pip install --user docker-compose
```

Now we need to add the current user to the docker user group so you can use containers. Type in the following.

```
sudo usermod -aG docker $USER && su $USER
```

Logout and log back in for the changes to be applied to your system.

We need to create a directory for the docker compose file. A compose file contains instructions to setup the container, just like instructions for the Lego set. Let's create a directory to put it in.

```
export DOWNLOAD_DIR=$HOME/greenbone-community-container && mkdir -p $DOWNLOAD_DIR
```

Note: The download directory in this tutorial is `/home/<user>/greenbone-community-container`. If you restart your computer at any point during the setup, the `DOWNLOAD_DIR` variable will lose its value. In such cases, replace `DOWNLOAD_DIR` with `./greenbone-community-container` in the commands.

Let's download the actual compose file.

```
cd $DOWNLOAD_DIR && curl -f -L <https://greenbone.github.io/docs/latest/_static/docker-compose-22.4.yml> -o docker-compose.yml
```

```
(mercury@pluto)-[~]
$ cd $DOWNLOAD_DIR && curl -f -L https://greenbone.github.io/docs/latest/_static/docker-compose-22.4.yml -o docker-compose.yml
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 4296 100 4296    0     0  771      0  0:00:05  0:00:05 --:--:--  929
```

Downloading the Docker Compose file

Once that's done, we can now setup (pull) the actual containers for OpenVAS.

```
docker-compose -f $DOWNLOAD_DIR/docker-compose.yml -p greenbone-community-edition pull
```

```
(mercury@pluto)-[~/greenbone-community-container]
$ docker-compose -f $DOWNLOAD_DIR/docker-compose.yml -p greenbone-community-edition up -d
Creating network "greenbone-community-edition_default" with the default driver
Creating volume "greenbone-community-edition_gpg_data_vol" with default driver
Creating volume "greenbone-community-edition_scap_data_vol" with default driver
Creating volume "greenbone-community-edition_cert_data_vol" with default driver
Creating volume "greenbone-community-edition_data_objects_vol" with default driver
Creating volume "greenbone-community-edition_gvmd_data_vol" with default driver
Creating volume "greenbone-community-edition_psql_data_vol" with default driver
Creating volume "greenbone-community-edition_vt_data_vol" with default driver
Creating volume "greenbone-community-edition_notus_data_vol" with default driver
Creating volume "greenbone-community-edition_psql_socket_vol" with default driver
Creating volume "greenbone-community-edition_gvmd_socket_vol" with default driver
Creating volume "greenbone-community-edition_ospd_openvas_socket_vol" with default driver
Creating volume "greenbone-community-edition_redis_socket_vol" with default driver
Pulling vulnerability-tests (greenbone/vulnerability-tests:...)
latest: Pulling from greenbone/vulnerability-tests
71d064a1ac7d: Pull complete
3948fedfef90: Extracting [=====] 45.12MB/61.12MB
2b29019d021d: Download complete
```

Pulling the OpenVAS containers

And now, we can start the containers. It should take a few minutes.

```
docker-compose -f $DOWNLOAD_DIR/docker-compose.yml -p greenbone-community-edition
up -d
```

```
Status: Downloaded newer image for greenbone/gvm-tools:latest
Creating greenbone-community-edition_pg-gvm_1 ... done
Creating greenbone-community-edition_cert-bund-data_1 ... done
Creating greenbone-community-edition_scap-data_1 ... done
Creating greenbone-community-edition_gpg-data_1 ... done
Creating greenbone-community-edition_mqtt-broker_1 ... done
Creating greenbone-community-edition_redis-server_1 ... done
Creating greenbone-community-edition_vulnerability-tests_1 ... done
Creating greenbone-community-edition_notus-data_1 ... done
Creating greenbone-community-edition_data-objects_1 ... done
Creating greenbone-community-edition_report-formats_1 ... done
Creating greenbone-community-edition_dfn-cert-data_1 ... done
Creating greenbone-community-edition_notus-scanner_1 ... done
Creating greenbone-community-edition_gvmd_1 ... done
Creating greenbone-community-edition_gsa_1 ... done
Creating greenbone-community-edition_ospd-openvas_1 ... done
Creating greenbone-community-edition_gvm-tools_1 ... done
```

Starting the Containers

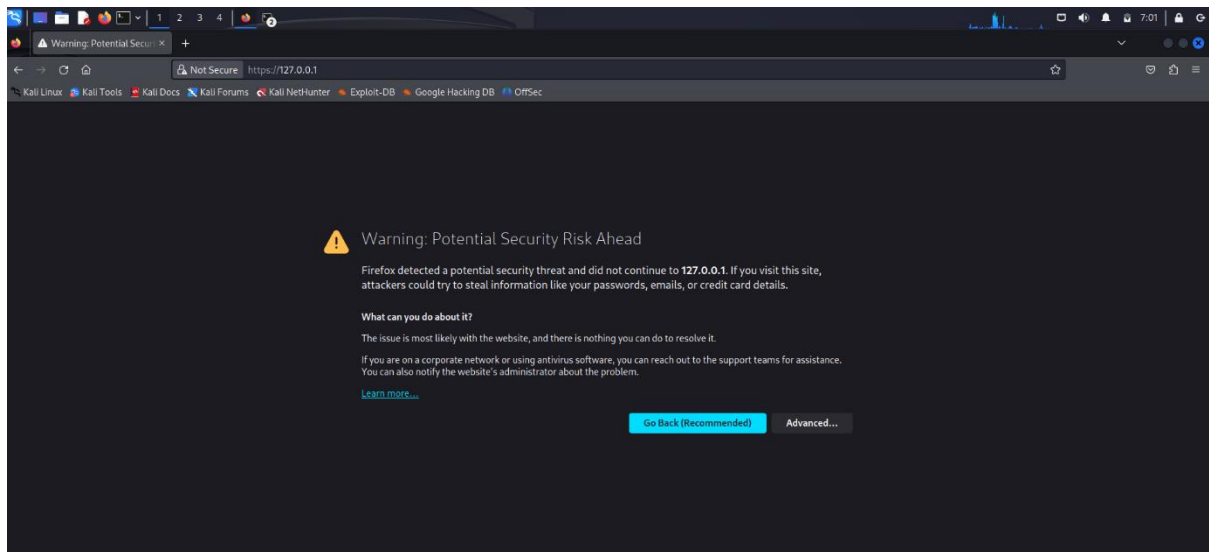
To open up the web interface for OpenVAS, run the command below.

```
xdg-open "<http://127.0.0.1:9392>" 2>/dev/null >/dev/null &
```

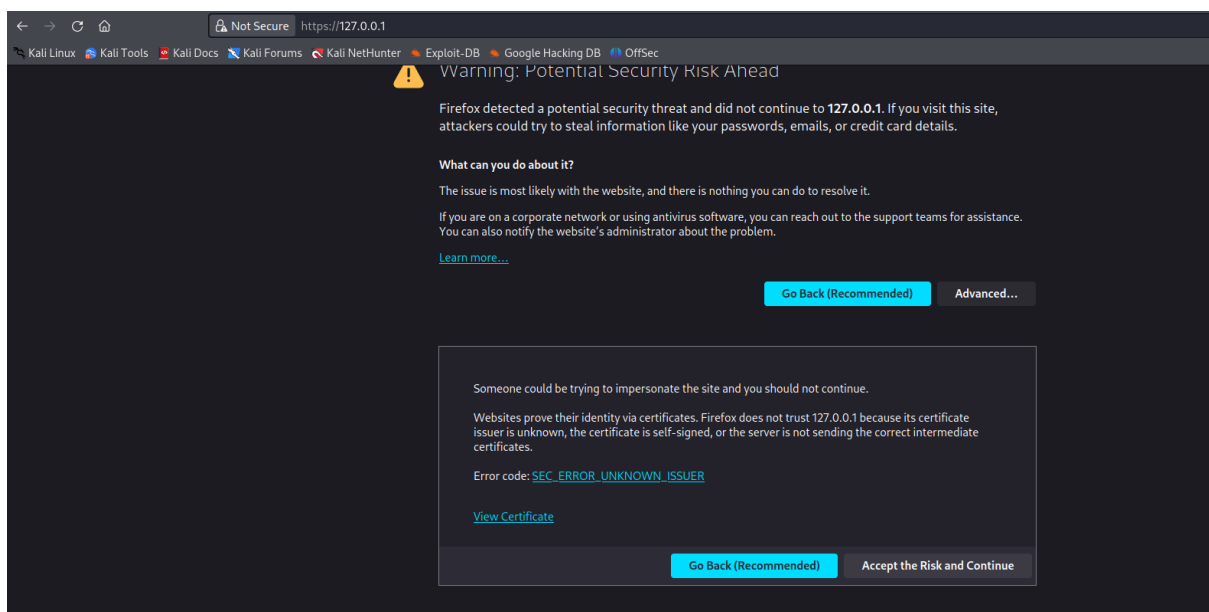
Alternatively, you could open up Firefox and type in the following.

```
<http://127.0.0.1:9392>
```

An interface just like the one below should pop up.



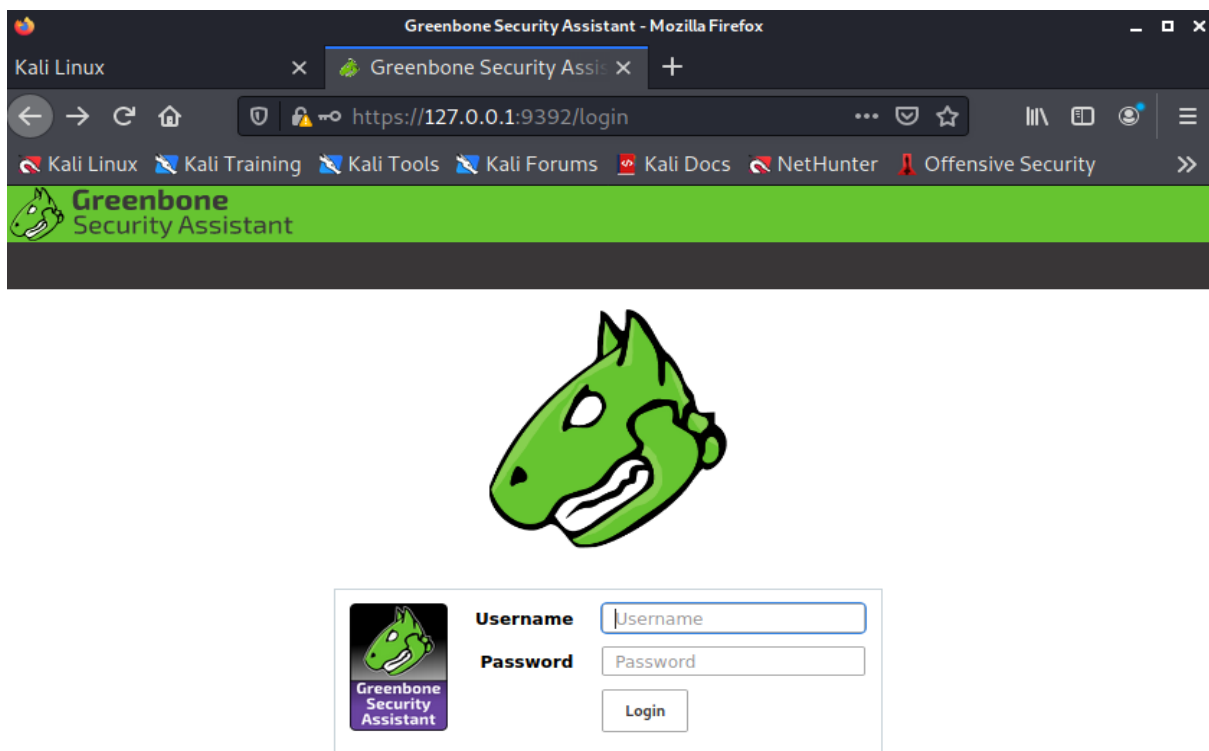
Click on Advanced



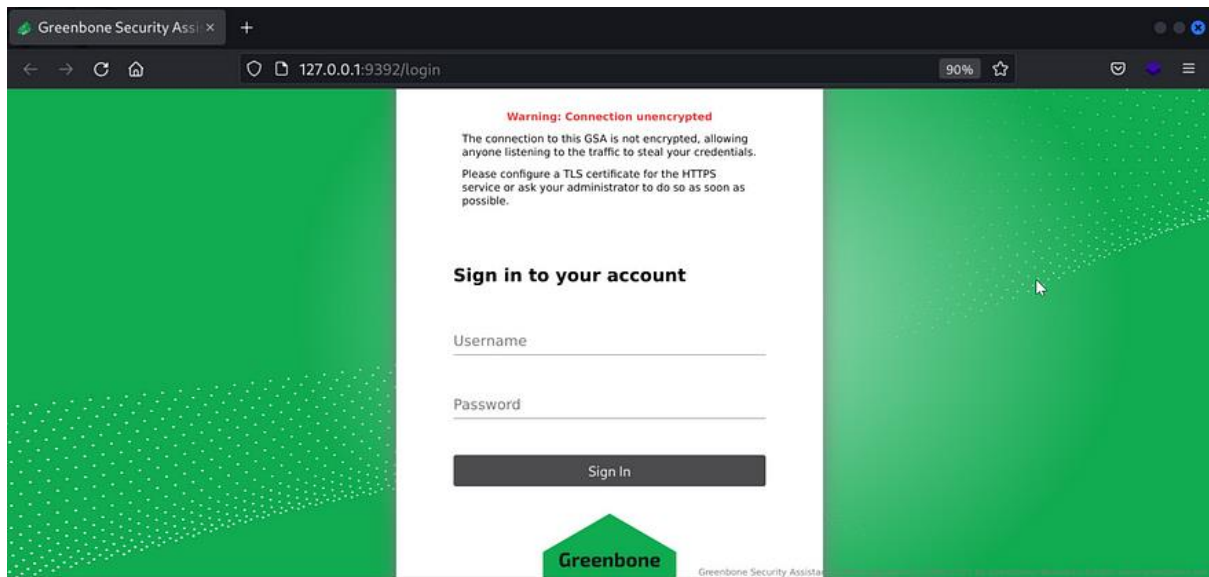
The next step is to accept the self-signed certificate warning and use the automatically generated admin credentials to login on to the web interface.

i.e., Then click on the [Accept the Risk and Continue](#)

You are redirected to the login page of Greenbone Security Assistance

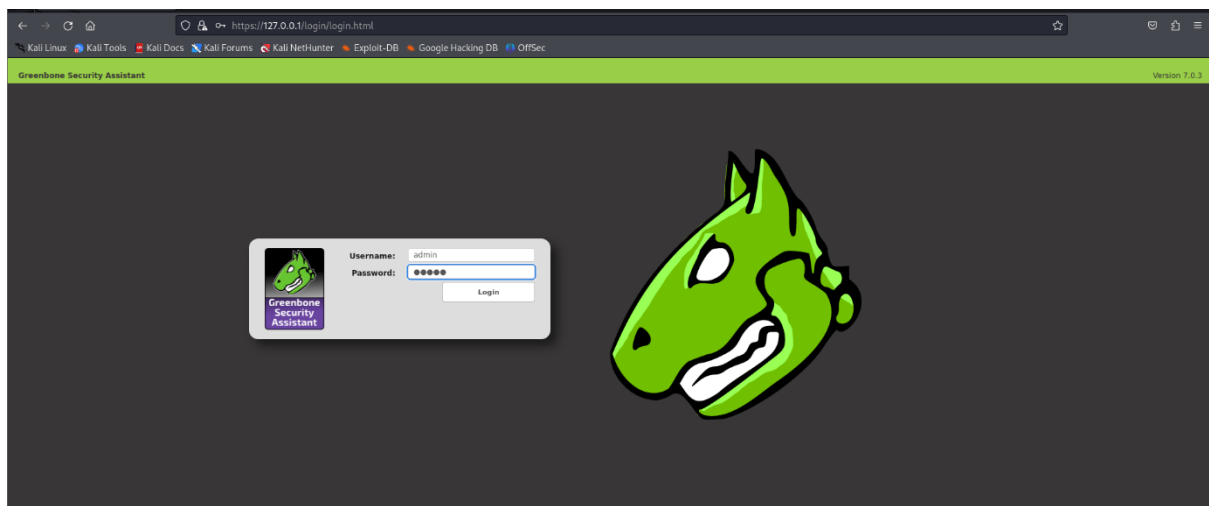


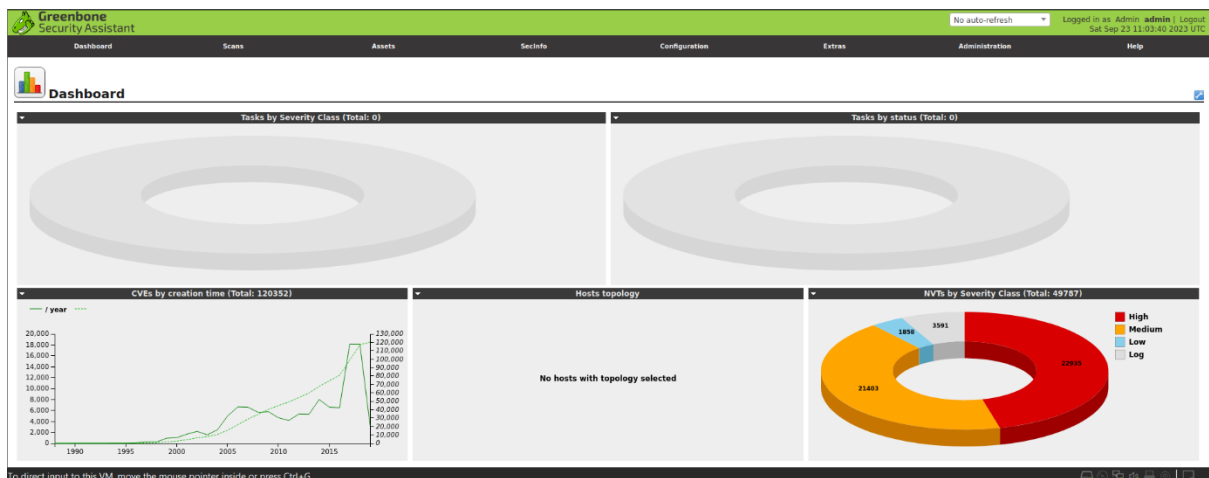
Either you will get above web page interface or below web page interface



The OpenVAS

The default username and password are `admin` and `admin` respectively. This is insecure so you might want to change them later.



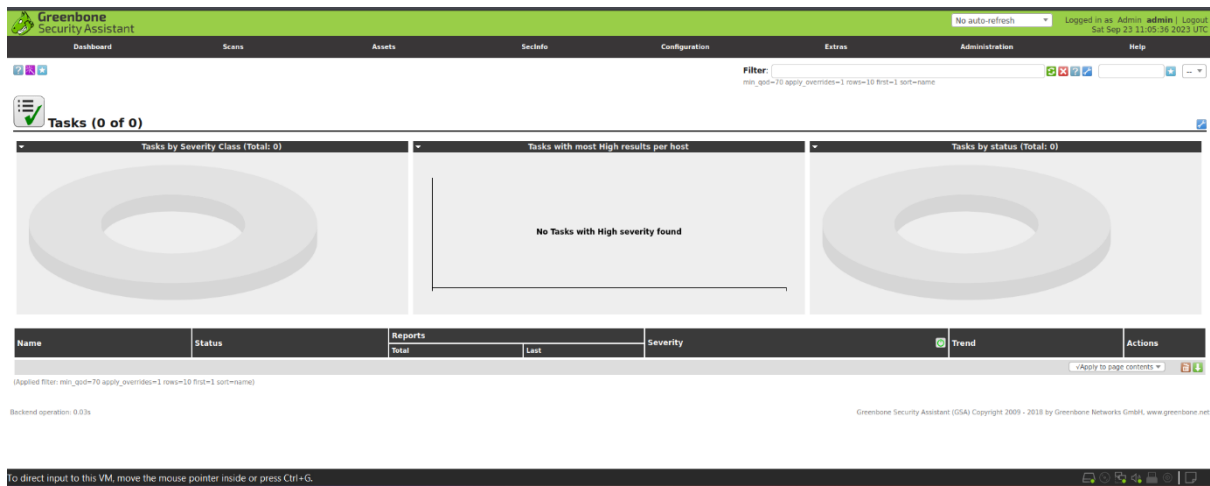


The OpenVAS Dashboard

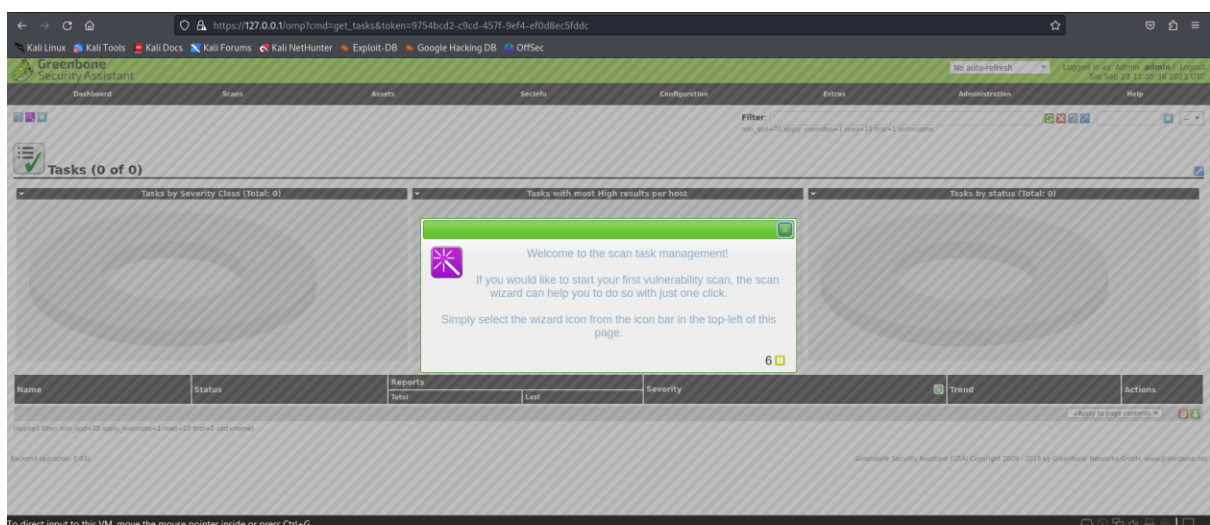
you should see something similar to the image above.

It's important to note that the scanner won't work immediately until all components are loaded. You can check this in this in the Feed Status under the Administration Tab. Depending on your hardware resources, it might take about 30 mins to a few hours.

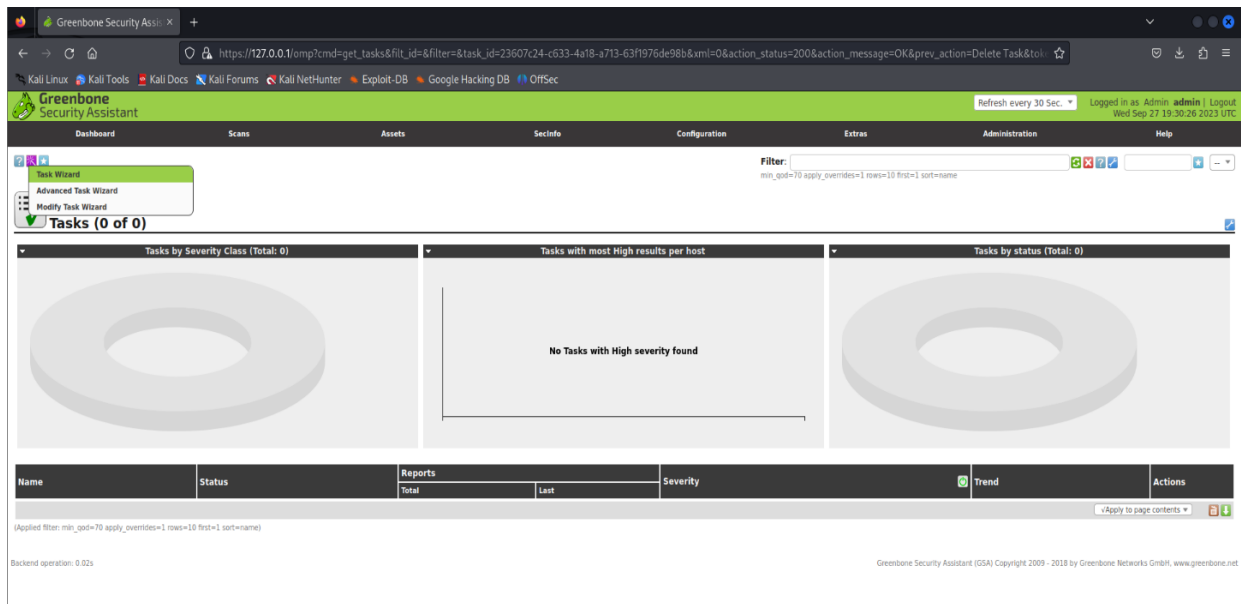
Once the feeds are updated, the column should show current on all tabs. Now to verify everything is working, we need to run a scan on our local host. Go to the Dashboard and click on the Wand icon at the top left to run a quick scan.



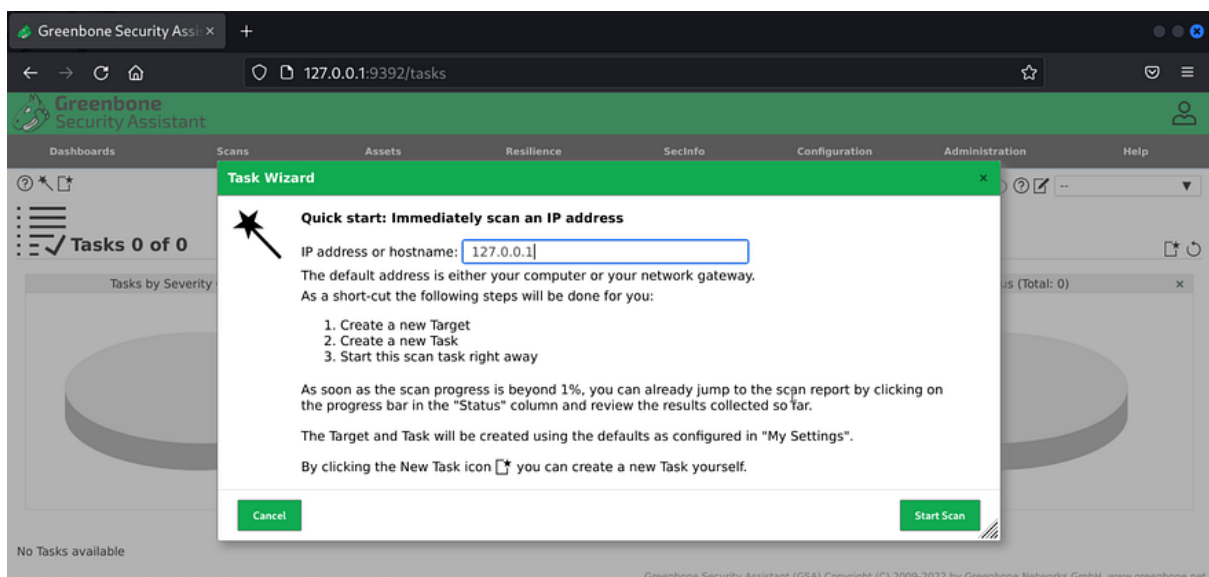
Initially we get Tasks (0 of 0)



For the first time it will count from the time 10 to zero to start shown in the above screenshot

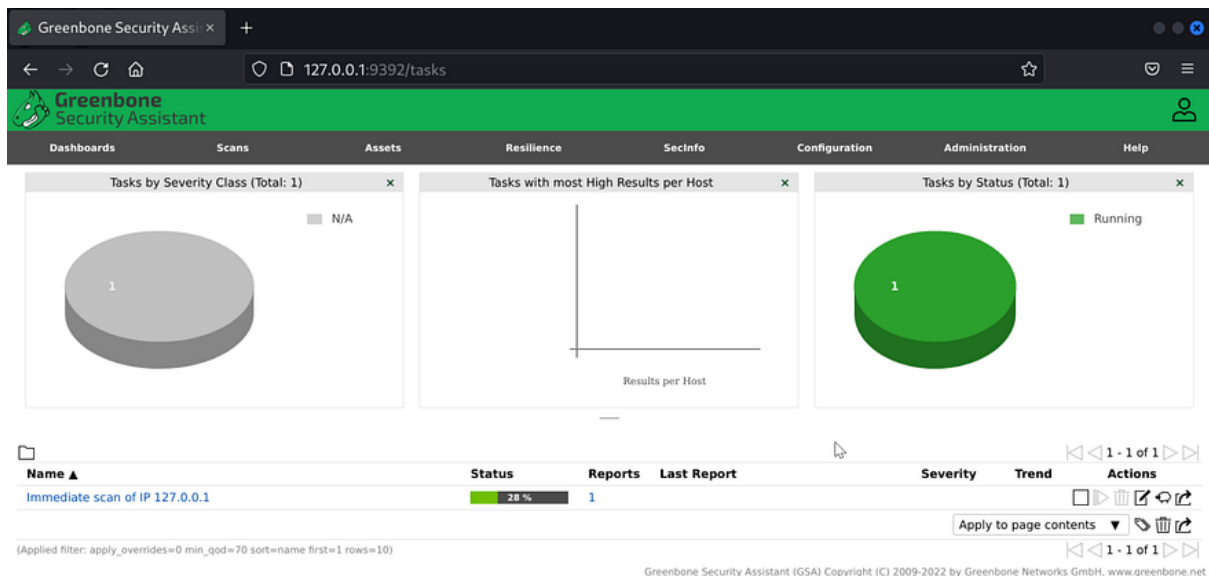


If you click on the wand icon it opens a small dialogue box with three different types of task wizards as seen the above screenshot



The Task Wizard

Put the address 127.0.0.1 and hit 'Start Scan'.



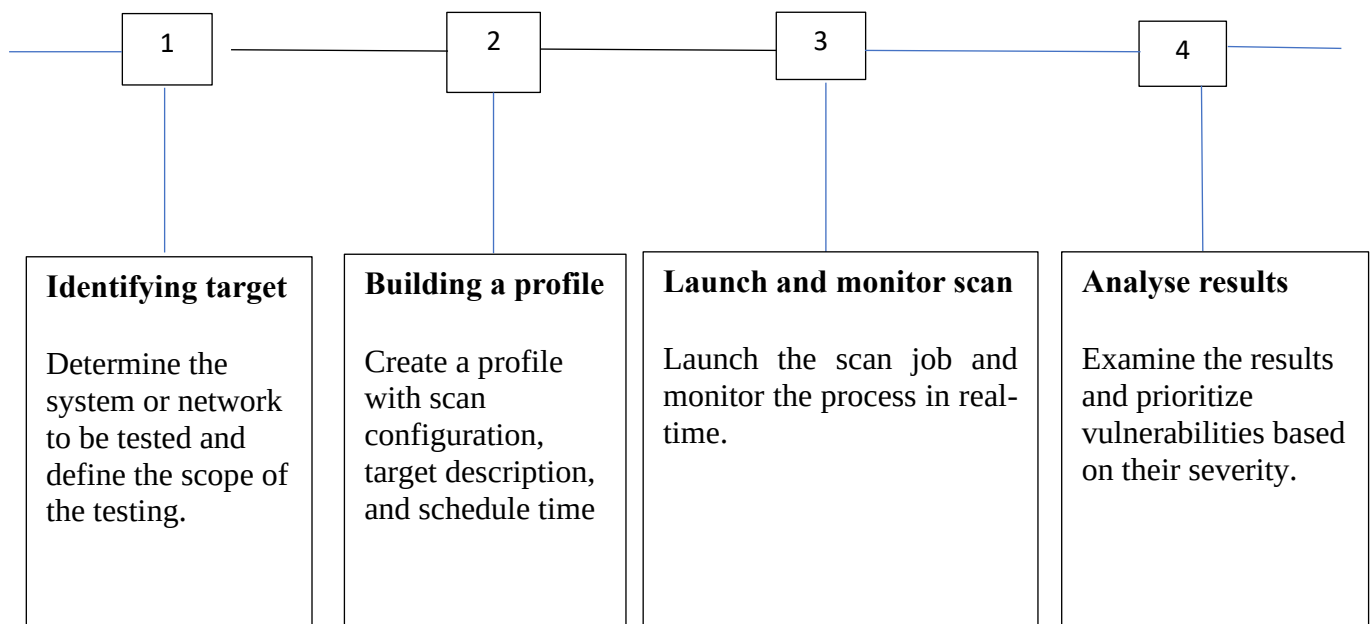
Running a task scan

If your Status bar goes past the 1% mark, give yourself a pat on the back. You have successfully installed OpenVAS.

METHODOLOGY

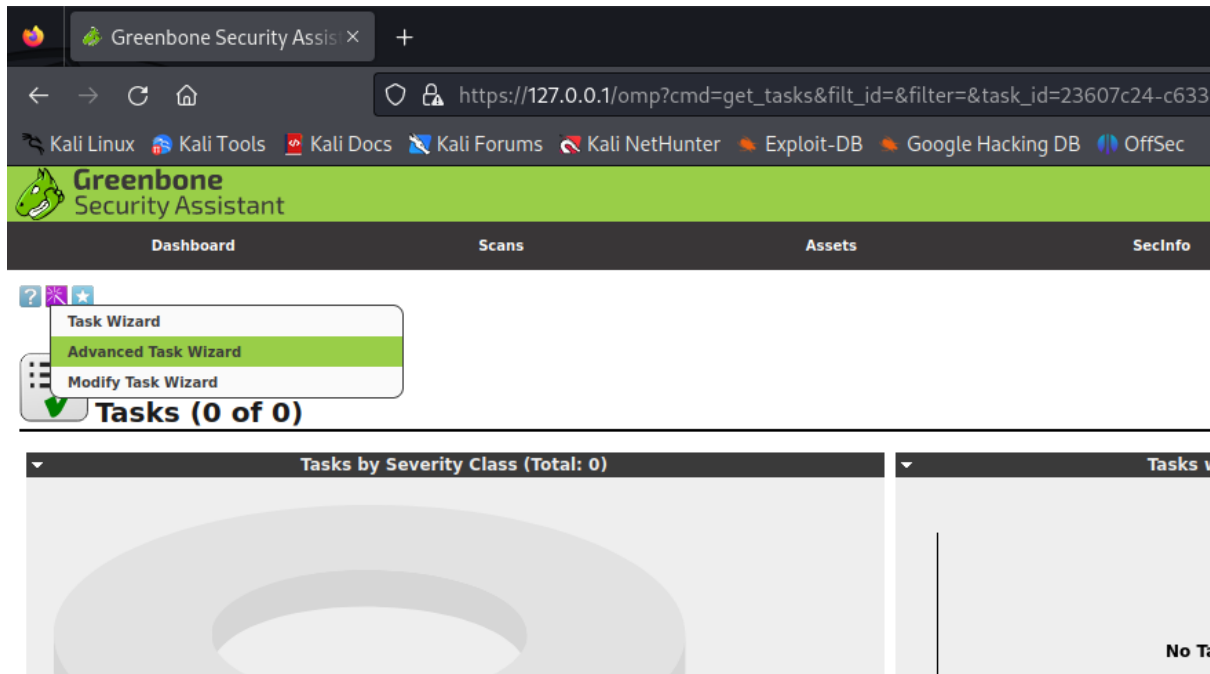
Methodology for Black Box Testing Using OpenVAS:

- Start OpenVAS and configure a new target for scanning.
- Schedule and start a scan of the target machine.
- Analyse the scan reports and identify the vulnerabilities.
- Use the OpenVAS exploitation modules to penetrate the system and extract data.

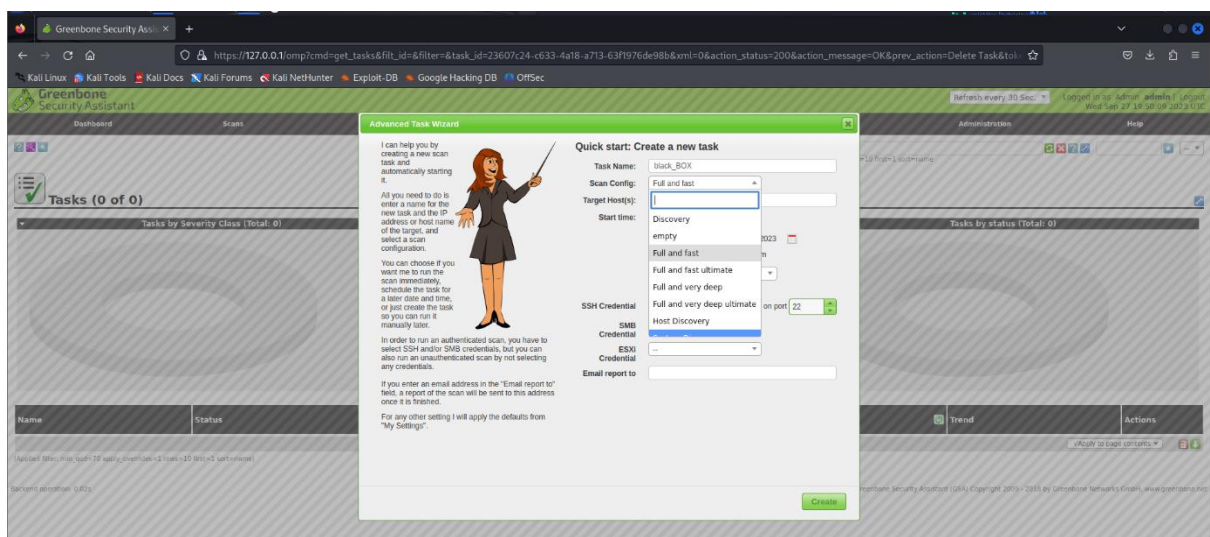


SCAN THE TARGET

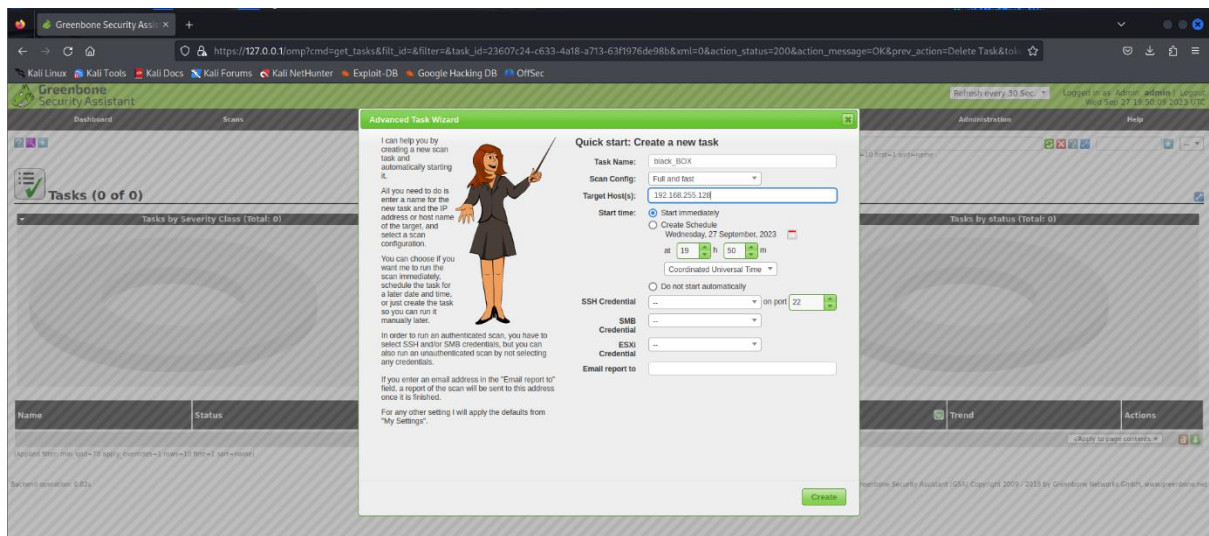
Go to the Dashboard and click on the Wand icon at the top left.



Choose Advanced Task Wizard then you will prompt for the creation of a new task.



Pick the full and very deep ultimate scan because it's time to get into this scan for real.



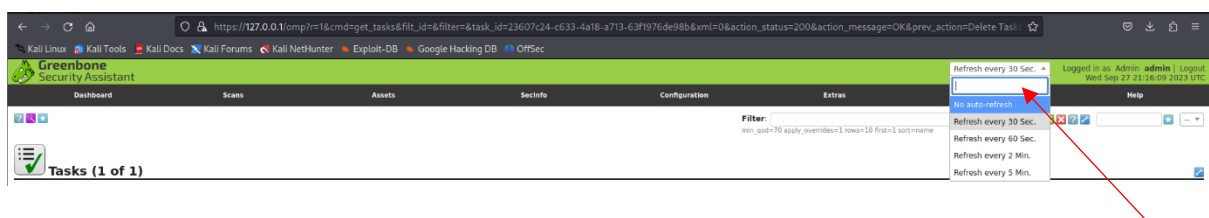
Here you can schedule the scan also based on your preference.

OpenVAS is capable of scanning remote systems, although it is not itself a port scanner. To do a complete job, experts recommend using a specialized network scanner such as Nmap. To pick the target IP Address.

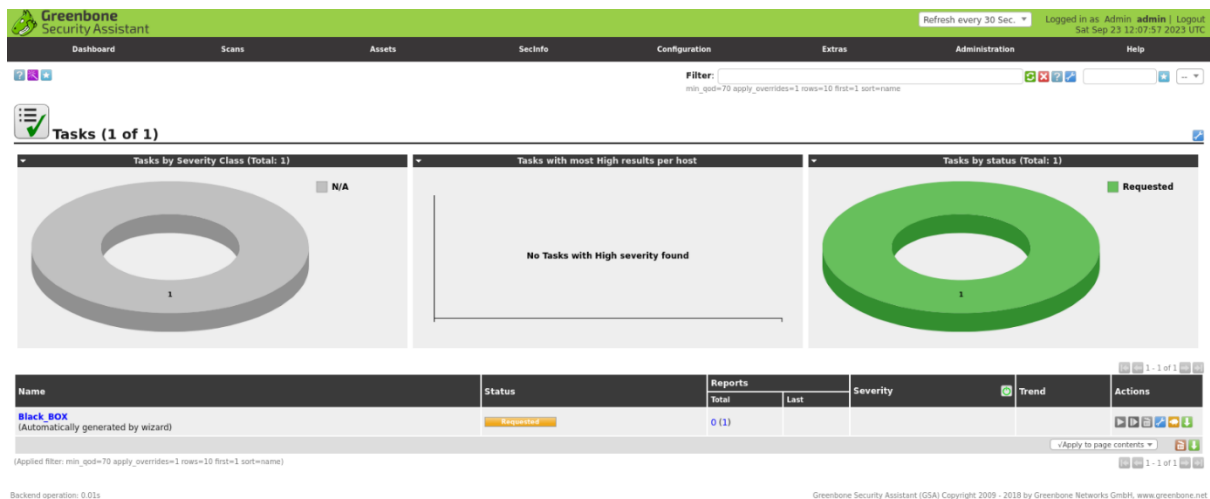
After clicking the create you our scan will be started.

Wait to change for status requested to 100.

Don't refresh the page, the OpenVAS will refresh the page for you for every 30 seconds.



Even We can able to change the refreshing rate of the OpenVAS.

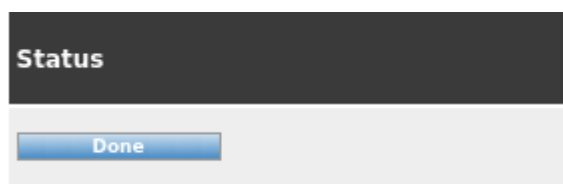


Once the Scan starts, sit back and wait for the results. This may take a while depending on the type of Scan you choose.



You can see the status in this box.

Wait for 100 percent.



We can see that the status is done now we can see the report that is generated by the OpenVAS Simply double click on done it will be redirected to report page.

RESULT AND CONCLUSION

The scan completed after a long time but it came back with some pretty good results. There are a lot of vulnerabilities which makes sense because Metasploitable is purposefully Vulnerable.



When the scan will be completed you are able to see the details of each vulnerability that has been found during the scanning procedure. As you can see in the image below in our report there are a lot of critical vulnerabilities. This is because we scanned a VM with metasploitable framework which is by default a vulnerable machine. At this point your employer needs to read your own report not the exported report of OpenVAS. So, the best practise is to write your own report listing your findings including suggestions and advices.

Dashboard

Scans

Assets

Secinfo

Configuration

Extras

Administration

Help

CSV Results

Filter:

autofp=0 apply_overrides=1 notes=1 overrides=1 result_hosts_reverse=severity levels=html min_qod=70

Users

Groups

Roles

LDAP

Radius

4233-456a-e27-15f9c82c4b0

3 12:29:54 2023

3 12:07:59 2023

Owner: admin

Report: Results (56 of 385)

1 - 56 of 56

Vulnerability	Severity	QoD	Host	Location	Actions
rexec Passwordless / Unencrypted Cleartext Login	10.0 (High)	80%	192.168.255.128	512/tcp	
OS End Of Life Detection	10.0 (High)	80%	192.168.255.128	general/tcp	
TWiki XSS and Command Execution Vulnerabilities	10.0 (High)	80%	192.168.255.128	80/tcp	
Distributed Ruby (dRuby/DRb) Multiple Remote Code Execution Vulnerabilities	10.0 (High)	99%	192.168.255.128	8787/tcp	
Possible Backdoor: Ingreslock	10.0 (High)	99%	192.168.255.128	1524/tcp	
DistCC Remote Code Execution Vulnerability	9.3 (High)	99%	192.168.255.128	3632/tcp	
VNC Brute Force Login	9.0 (High)	95%	192.168.255.128	5900/tcp	
MySQL / MariaDB weak password	9.0 (High)	95%	192.168.255.128	3306/tcp	
PostgreSQL weak password	9.0 (High)	99%	192.168.255.128	5432/tcp	
rsh Unencrypted Cleartext Login	7.5 (High)	80%	192.168.255.128	514/tcp	
rlogin Passwordless / Unencrypted Cleartext Login	7.5 (High)	70%	192.168.255.128	513/tcp	
Tiki Wiki CMS Groupware < 4.2 Multiple Unspecified Vulnerabilities	7.5 (High)	80%	192.168.255.128	80/tcp	
phpinfo() output Reporting	7.5 (High)	80%	192.168.255.128	80/tcp	
Test HTTP dangerous methods	7.5 (High)	99%	192.168.255.128	80/tcp	
Check for Backdoor in UnrealIRCd	7.5 (High)	70%	192.168.255.128	6667/tcp	
vsftpd Compromised Source Packages Backdoor Vulnerability	7.5 (High)	99%	192.168.255.128	6200/tcp	
vsftpd Compromised Source Packages Backdoor Vulnerability	7.5 (High)	99%	192.168.255.128	21/tcp	
PHP-CGI-based setups vulnerability when parsing query string parameters from php files.	7.5 (High)	95%	192.168.255.128	80/tcp	
SSH Brute Force Logins With Default Credentials Reporting	7.5 (High)	95%	192.168.255.128	22/tcp	
UnrealIRCd Authentication Spoofing Vulnerability	6.8 (Medium)	80%	192.168.255.128	6667/tcp	
TWiki Cross-Site Request Forgery Vulnerability - Sep10	6.8 (Medium)	80%	192.168.255.128	80/tcp	
Multiple Vendors STARTTLS Implementation Plaintext Arbitrary Command Injection Vulnerability	6.8 (Medium)	99%	192.168.255.128	25/tcp	
SSL/TLS: OpenSSL CCS Man in the Middle Security Bypass Vulnerability	6.8 (Medium)	70%	192.168.255.128	5432/tcp	
Tiki Wiki CMS Groupware < 17.2 SQL Injection Vulnerability	6.5 (Medium)	80%	192.168.255.128	80/tcp	
Anonymous FTP Login Reporting	6.4 (Medium)	80%	192.168.255.128	21/tcp	
TWiki Cross-Site Request Forgery Vulnerability	6.0 (Medium)	80%	192.168.255.128	80/tcp	
HTTP Debugging Methods (TRACE/TRACK) Enabled	5.0 (Medium)	99%	192.168.255.128	80/tcp	
Check if Mailserver answer to VRFY and EXPN requests	5.0 (Medium)	99%	192.168.255.128	25/tcp	
SSL/TLS: Certificate Expired	5.0 (Medium)	99%	192.168.255.128	25/tcp	
SSL/TLS: Certificate Expired	5.0 (Medium)	99%	192.168.255.128	5432/tcp	
/doc directory browsable	5.0 (Medium)	80%	192.168.255.128	80/tcp	
Tiki Wiki CMS Groupware Input Sanitation Weakness Vulnerability	5.0 (Medium)	80%	192.168.255.128	80/tcp	
Tiki Wiki CMS Groupware 'fixedURLData' Local File Inclusion Vulnerability	5.0 (Medium)	80%	192.168.255.128	80/tcp	
awiki Multiple Local File Include Vulnerabilities	5.0 (Medium)	99%	192.168.255.128	80/tcp	
VNC Server Unencrypted Data Transmission	4.8 (Medium)	70%	192.168.255.128	5900/tcp	
Telnet Unencrypted Cleartext Login	4.8 (Medium)	70%	192.168.255.128	23/tcp	
FTP Unencrypted Cleartext Login	4.8 (Medium)	70%	192.168.255.128	2121/tcp	
FTP Unencrypted Cleartext Login	4.8 (Medium)	70%	192.168.255.128	21/tcp	
Cleartext Transmission of Sensitive Information via HTTP	4.8 (Medium)	80%	192.168.255.128	80/tcp	
SSH Weak Encryption Algorithms Supported	4.3 (Medium)	95%	192.168.255.128	22/tcp	
SSL/TLS: Deprecated SSLv2 and SSLv3 Protocol Detection	4.3 (Medium)	98%	192.168.255.128	5432/tcp	
SSL/TLS: Deprecated SSLv2 and SSLv3 Protocol Detection	4.3 (Medium)	98%	192.168.255.128	25/tcp	
SSL/TLS: Report Weak Cipher Suites	4.3 (Medium)	98%	192.168.255.128	5432/tcp	
SSL/TLS: RSA Temporary Key Handling 'RSA_EXPORT' Downgrade Issue (FREAK)	4.3 (Medium)	80%	192.168.255.128	25/tcp	
SSL/TLS: SSLv3 Protocol CBC Cipher Suites Information Disclosure Vulnerability (POODLE)	4.3 (Medium)	80%	192.168.255.128	5432/tcp	
SSL/TLS: SSLv3 Protocol CBC Cipher Suites Information Disclosure Vulnerability (POODLE)	4.3 (Medium)	80%	192.168.255.128	25/tcp	
SSL/TLS: 'DHE_EXPORT' Man in the Middle Security Bypass Vulnerability (Logjam)	4.3 (Medium)	80%	192.168.255.128	25/tcp	
TWiki < 6.1.0 XSS Vulnerability	4.3 (Medium)	80%	192.168.255.128	80/tcp	
Apache HTTP Server 'httpOnly' Cookie Information Disclosure Vulnerability	4.3 (Medium)	99%	192.168.255.128	80/tcp	
phpMyAdmin 'error.php' Cross Site Scripting Vulnerability	4.3 (Medium)	99%	192.168.255.128	80/tcp	
SSL/TLS: Diffie-Hellman Key Exchange Insufficient DH Group Strength Vulnerability	4.0 (Medium)	80%	192.168.255.128	5432/tcp	
SSL/TLS: Diffie-Hellman Key Exchange Insufficient DH Group Strength Vulnerability	4.0 (Medium)	80%	192.168.255.128	25/tcp	
SSL/TLS: Certificate Signed Using A Weak Signature Algorithm	4.0 (Medium)	80%	192.168.255.128	5432/tcp	
Tiki Wiki CMS Groupware XSS Vulnerability	3.5 (Low)	80%	192.168.255.128	80/tcp	
SSH Weak MAC Algorithms Supported	2.6 (Low)	95%	192.168.255.128	22/tcp	
TCP timestamps	2.6 (Low)	80%	192.168.255.128	general/tcp	

(Applied filter: autofp=0 apply_overrides=1 notes=1 overrides=1 result_hosts_only=1 first=1 rows=100 sort=reverse=severity levels=html min_qod=70)

1 - 56 of 56

Report for the scan:

The screenshot shows the Greenbone Security Assistant (GSA) interface. At the top, there's a green header with the logo and navigation tabs: Dashboard, Scans, Assets, SecInfo, Configuration, Extras, Administration, and Help. The user is logged in as 'Admin: admin' on 'Sat Sep 23 12:34:06 2023 UTC'. Below the header, there's a section titled 'Result: FTP Unencrypted Cleartext Login'. It includes a table with columns: Vulnerability, Severity, QoD, Host, Location, and Actions. The vulnerability is 'FTP Unencrypted Cleartext Login' with a severity of 'Medium' and a QoD of '70%'. The host is '192.168.255.128' and the location is '2121/tcp'. Below the table, there's a 'Summary' section, a 'Vulnerability Detection Result' section, an 'Impact' section, a 'Solution' section, and a 'Vulnerability Detection Method' section. At the bottom, there's a 'User Tags (none)' section.

Vulnerability	Severity	QoD	Host	Location	Actions
FTP Unencrypted Cleartext Login	Medium	70%	192.168.255.128	2121/tcp	

We can analyse each vulnerability by click on it.

It also provides the mitigation steps also.

The screenshot shows the 'Dashboard' and 'Scans' tabs. Below them, there's a dropdown menu with 'Anonymous XML' selected. To the right of the dropdown are three icons: a green down arrow, a red up arrow, and a green checkmark. A 'Done' button is also visible.

On the top we can select the option to save the report.

The screenshot shows a dropdown menu with 'PDF' selected. To the right of the dropdown is a green down arrow icon.

Result can also be downloaded in HTML/CSV/PDF formats.

Select the required format and click on the green down arrow to download the report and now you can close the OpenVAS.

To view above performed report scan [CLICK HERE](#).

Here is the summary and contents of scan report

Scan Report

September 23, 2023

Summary

This document reports on the results of an automatic security scan. All dates are displayed using the timezone "Coordinated Universal Time", which is abbreviated "UTC". The task was "BlackOX". The scan started at Sat Sep 23 12:07:59 2023 UTC and ended at Sat Sep 23 12:29:54 2023 UTC. The report first summarises the results found. Then, for each host, the

Contents

1	Result Overview	2
1.1	Host Authentications	2
2	Results per Host	2
2.1	192.168.255.128	2
2.1.1	High 21/tcp	3
2.1.2	High general/tcp	4
2.1.3	High 513/tcp	5
2.1.4	High 3306/tcp	5
2.1.5	High 5900/tcp	6
2.1.6	High 22/tcp	7
2.1.7	High 8787/tcp	7
2.1.8	High 5432/tcp	9
2.1.9	High 512/tcp	9
2.1.10	High 6200/tcp	10
2.1.11	High 3632/tcp	11
2.1.12	High 1524/tcp	12
2.1.13	High 6667/tcp	12
2.1.14	High 514/tcp	13
2.1.15	High 80/tcp	14
2.1.16	Medium 21/tcp	19
2.1.17	Medium 25/tcp	20
2.1.18	Medium 5900/tcp	28

<i>CONTENTS</i>		2
2.1.19	Medium 22/tcp	29
2.1.20	Medium 5432/tcp	30
2.1.21	Medium 23/tcp	37
2.1.22	Medium 6667/tcp	37
2.1.23	Medium 2121/tcp	39
2.1.24	Medium 80/tcp	39
2.1.25	Low general/tcp	51
2.1.26	Low 22/tcp	52
2.1.27	Low 80/tcp	52

Result Overview

Host	High	Medium	Low	Log	False Positive
192.168.255.128	19	34	3	0	0
Total: 1	19	34	3	0	0

Vendor security updates are not trusted.

Overrides are on. When a result has an override, this report uses the threat of the override.

Information on overrides is excluded from the report.

Notes are included in the report.

This report might not show details of all issues that were found.

It only lists hosts that produced issues.

Issues with the threat level "Log" are not shown.

Issues with the threat level "Debug" are not shown.

Issues with the threat level "False Positive" are not shown.

Only results with a minimum QoD of 70 are shown.

This report contains all 56 results selected by the filtering described above. Before filtering there were 385 results.

Host Authentications

Host	Protocol	Result	Port/User
192.168.255.128	SMB	Success	Protocol SMB, Port 445, User

Results per Host

192.168.255.128

Host scan start Sat Sep 23 12:08:11 2023 UTC

Host scan end Sat Sep 23 12:29:53 2023 UTC

Service (Port)	Threat Level
21/tcp	High
general/tcp	High
513/tcp	High
3306/tcp	High
5900/tcp	High
22/tcp	High
8787/tcp	High
5432/tcp	High
512/tcp	High
6200/tcp	High

... (continues) ...

... (continued) ...

Service (Port)	Threat Level
3632/tcp	High
1524/tcp	High
6667/tcp	High
514/tcp	High
80/tcp	High
21/tcp	Medium
25/tcp	Medium
5900/tcp	Medium
22/tcp	Medium
5432/tcp	Medium
23/tcp	Medium
6667/tcp	Medium
2121/tcp	Medium
80/tcp	Medium
general/tcp	Low
22/tcp	Low
80/tcp	Low

If you take one of the high severity vulnerabilities from the above scan report It gives summary, vulnerability detection result, solution to fix it and how it detected vulnerability. Likewise in the scan it listed many vulnerabilities.

High 22/tcp

High (CVSS: 7.5) NVT: SSH Brute Force Logins With Default Credentials Reporting
Summary It was possible to login into the remote SSH server using default credentials. As the NVT 'SSH Brute Force Logins with default Credentials' (OID: 1.3.6.1.4.1.25623.1.0.108013) might run into a timeout the actual reporting of this vulnerability takes place in this NVT instead. The script preference 'Report timeout' allows you to configure if such an timeout is reported.
Vulnerability Detection Result It was possible to login with the following credentials <User>:<Password> msfadmin:msfadmin user:user
Solution Solution type: Mitigation Change the password as soon as possible.
Vulnerability Detection Method Try to login with a number of known default credentials via the SSH protocol. Details: SSH Brute Force Logins With Default Credentials Reporting OID:1.3.6.1.4.1.25623.1.0.103239 Version used: \$Revision: 13568 \$

[| return to 192.168.255.128 |](#)

REFERENCES

- <https://www.kali.org/get-kali/#kali-platforms>
- <https://openvas.org/>
- <https://www.vmware.com/in/products/workstation-pro/workstation-pro-evaluation.html>
- <https://hub.docker.com/r/mikesplain/openvas/>
- https://youtube.com/playlist?list=PLw2kw_XA_nSHz0eBb-fW3BYDzfiQI2_8w&si=eR-4yE96RkpmX061
- <https://www.acunetix.com/blog/articles/black-box-security-testing/>
- **GOOGLE**

GLOSSARY

OpenVAS (Open Vulnerability Assessment Scanner, originally known as GNessUs) is the scanner component of Greenbone Vulnerability Management (GVM), a software framework of several services and tools offering vulnerability scanning and vulnerability management

CVSS: An evaluation of the potential impact and likelihood of vulnerabilities being exploited, often quantified using tools like the Common Vulnerability Scoring System (CVSS).

Security Posture: An organization's overall security readiness and resilience, reflecting its capacity to defend against, respond to, and recover from security threats and vulnerabilities.

Penetration Testing: Penetration testing, often called "pen testing," is a proactive security assessment where authorized experts simulate real-world cyberattacks on a system, network, or application to identify vulnerabilities and assess security weaknesses. The goal is to uncover potential risks, allowing organizations to strengthen their defences and protect against malicious threats.

Launching Pad: Initiation or Execution machine for an attack

Proactive Approach: A proactive approach is about taking anticipatory actions or measures to prevent issues or seize opportunities before they occur, rather than reacting to them after they happen. It's all about being ahead of the curve and staying prepared.

Stakeholders: Stakeholders are individuals, groups, or organizations with an interest or involvement in a particular project, business, or decision. They can influence or be affected by the outcome and often have specific concerns or objectives related to it.

VM: A virtual machine (VM) is a software-based emulation of a physical computer. It allows you to run multiple operating systems or environments on a single physical machine, providing flexibility, isolation, and efficient resource utilization.

Nmap: Nmap (Network Mapper) is a widely used network scanning tool that helps discover and map devices and services on a network. It's a versatile tool for security assessments and network exploration, offering insights into network topology and potential vulnerabilities.

APPENDIX

In this appendix, we provide supplementary information and resources related to the project

Black box security testing is a critical process for assessing the security of a host machine or network. OpenVAS (Open Vulnerability Assessment System) is a popular open-source vulnerability scanner that can help identify potential security weaknesses and vulnerabilities. Below, you'll find a step-by-step guide on how to perform black box security testing using OpenVAS on a host machine.

Note: Before you begin, ensure that you have OpenVAS installed and configured on your testing machine.

❖ Preparation:

- Make sure your host machine and network are properly configured and isolated from your production environment to avoid any unintended consequences.

❖ Scoping:

- Define the scope of your black box testing, including the IP addresses or domains you want to test.

- Identify the objectives and goals of your testing to ensure a focused assessment.

❖ Network Scanning:

- Use OpenVAS to perform an initial network scan to identify live hosts and open ports on the target machine. You can use the ``openvas-scaphdata-sync`` and ``openvas-certdata-sync`` commands to update vulnerability databases.

```
sudo openvas-scaphdata-sync
```

```
sudo openvas-certdata-sync
```

- Use ``nmap`` or similar tools to perform a detailed network scan if necessary.

❖ Target Enumeration:

- Gather information about the target, such as domain names, subdomains, and IP addresses, using tools like ``dnsenum``, ``theHarvester``, or manual techniques.

❖ Vulnerability Scanning:

- Configure OpenVAS to scan the target host(s) for vulnerabilities. You can use the OpenVAS web interface or command-line tools like ``omp`` to start scans.

- omp -u <username> -w <password> -G

- Create or import target hosts and configure scan policies based on your testing objectives.

❖ **Analysis and Reporting:**

- Once the scans are complete, review the results to identify vulnerabilities and weaknesses in the target system.

- Prioritize vulnerabilities based on severity, potential impact, and exploitability.

❖ **Exploitation (Optional):**

- If authorized and part of your testing scope, attempt to exploit vulnerabilities to assess the system's resilience against attacks. Exercise caution and follow ethical hacking guidelines.

❖ **Documentation:**

- Document all findings, including the vulnerabilities discovered, their severity, and recommendations for mitigation.

- Include detailed steps to reproduce any discovered vulnerabilities.

❖ **Reporting:**

- Generate a comprehensive report of the security assessment, including an executive summary, detailed findings, risk assessment, and recommendations for remediation.

- Share the report with relevant stakeholders and management.

❖ **Remediation:**

- Work with the system administrators or IT teams to address and mitigate the identified vulnerabilities.

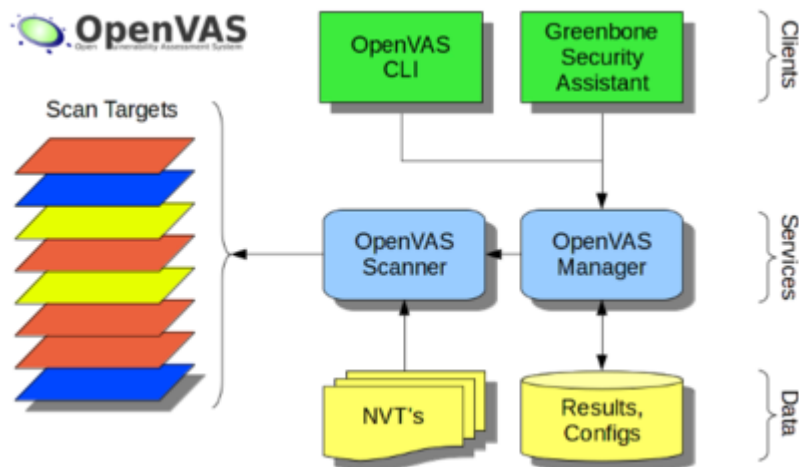
- Verify that the remediation actions effectively resolve the issues.

❖ **Retesting (Optional):**

- Conduct follow-up scans to ensure that the vulnerabilities have been successfully mitigated.

❖ **Continuous Monitoring:**

- Implement continuous monitoring and periodic security assessments to maintain the security posture of the host machine and network.



The OpenVAS 8 Structure

Accessing the OpenVAS Web Interface

The OpenVAS Web Interface (gsad) runs on TCP port 9392. However, depending on your installation, it could also be listening on TCP 443. After installation this can be confirmed by checking the listening ports on your system.

Active Internet connections (servers and established)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State	PID/Program name
tcp	0	0	127.0.0.1:6379	0.0.0.0:*	LISTEN	3692/redis-server 1
tcp	0	0	0.0.0.0:9391	0.0.0.0:*	LISTEN	13806/openvassd: Wa
tcp	0	0	0.0.0.0:1337	0.0.0.0:*	LISTEN	3656/sshd
tcp6	0	0	::: 9390	::: *	LISTEN	13804/openvasmd
tcp6	0	0	::: 443	::: *	LISTEN	28020/gsad

From these ports, we can see in our installation gsad is running on port 443. The OpenVAS Scanner (openvassd) is running on TCP Port 9391 and the OpenVAS Manager (openvasmd) is

running on TCP port 9390. Lastly is the redis-server on TCP 6379. This database is used to store temporary meta data during active scanning.

OpenVAS is a highly capable and powerful vulnerability testing solution. Hopefully, this tutorial and included tips will prove useful to users out there. The team at Greenbone Networks, along with the community and other supporters, have done a fantastic job building OpenVAS. The latest version has a range of excellent new features and optimisations that have been a big step forward.

For some of these items, I have only scratched the surface. Use the information here as pointers to allow you to dig deeper into the OpenVAS system.

There is a lot of good information available on the OpenVAS site and from the mailing list archive. Another often overlooked location for further information is the man pages for the `openvassd` and `openvasmd` servers.